Markov Decision Processes

CSC 480: Artificial Intelligence, Spring 2026

I Part I: Markov Decision Processes

Q1: [1] (a) Define a Markov Decision Process. List all five components, give the mathematical notation for each, and explain what each one represents.

An MDP is a formal model of sequential decision-making under uncertainty. The five components are:

- S: the set of all possible *states* the agent can be in.
- A: the set of all possible *actions* the agent can take.
- $T(s, a, s')$: the *transition model*, the probability of landing in state s' after taking action a in state s.
- $R(s, a, s')$: the *reward function*, the immediate numeric payoff received when transitioning from s to s' via action a.
- $\gamma \in [0, 1)$: the *discount factor*, which controls how much the agent values future rewards relative to immediate ones.

(b) Then write the distribution constraint that the transition model must satisfy for every state action pair (s, a), and explain why it is necessary.

For every state-action pair (s, a):

$$\sum_{s'} T(s, a, s') = 1 \tag{1}$$

This is necessary because $T(s, a, \cdot)$ is a probability distribution over next states. Every action must lead *somewhere* with total probability 1, the agent cannot disappear into the void, and probability that sums to less than 1 would leave unaccounted outcomes.

Q2: State the Markov property formally. What does it say about the relationship between the future state and the full history of past states and actions? Why does this property make MDPs tractable? What concept from Bayes Networks captures the same idea? [2]

Formally:

$$P(S_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, ..., S_0, A_0) = P(S_{t+1} \mid S_t, A_t) \tag{2}$$

The next state depends only on the *current* state and action, not on the full history of how you got there. This makes MDPs tractable because the agent only needs to remember the current state to act optimally; there is no need to

Contents

[1] 08-MDP-Bellman, Pages 1-2

[2] 08-MDP-Bellman, Page 3

store or reason over an exponentially growing history. The equivalent concept in Bayes Networks is *conditional independence* (or d-separation): $S_{\{t+1\}}$ is conditionally independent of all earlier states and actions given S_t and A_t.

Q3: What is a *policy*? How does it differ from a *plan* as computed in a search problem? Explain the difference between the two as it pretains to MDP solution spaces. [3]

[3] 08-MDP-Bellman, Page 4

A *policy* $\pi : S \to A$ is a function that specifies which action to take in every possible state. A *plan* from search is a fixed sequence of actions computed for a deterministic path (like a grocery list, do step 1, then step 2, etc.).

In a stochastic MDP, you don't know which state you'll end up in after an action, so a fixed action sequence breaks down immediately when the environment surprises you. A policy handles this by providing a contingency plan for every state: wherever the MDP dumps you, the policy tells you what to do. Plans are open-loop; policies are closed-loop.

Q4: Consider the following MDP describing a roomba operating in a home. The robot can be in one of four states: **Clean** (the floor is tidy), **Messy** (debris has accumulated), **Jammed** (wedged under furniture but potentially recoverable), or **Broken** (the robot has been irreparably damaged). The full transition and reward model is: [4]

[4] I made some abbreviations to make it all fit in the table

State	**Action**	P' **(Clean)**	P' **(Messy)**	P' **(Jam'd)**	P' **(Broke)**	**Reward**
Clean	Patrol	0.7	0.3	0.0	0.0	+3
Clean	Charge	1.0	0.0	0.0	0.0	+1
Messy	Vacuum	0.6	0.35	0.05	0.0	+5
Messy	Explore	0.4	0.3	0.3	0.0	+8
Jammed	Reset	0.0	1.0	0.0	0.0	−5
Jammed	Force	0.5	0.0	0.0	0.5	+1
Broken	Wait	0.0	0.0	0.0	1.0	−20

Explore sends the robot into unexplored territory: 40% of the time it finds a shortcut to a cleaner configuration, 30% of the time it finishes where it started, and 30% of the time it wedges itself under furniture. When *jammed* the robot has two recovery options. *Reset* where it carefully backs out, always landing in Messy (rewarded with −5 for the wasted effort) and *Force* where it tries to muscle free. 50% of the time it succeeds and lands in Clean, but 50% of the time it destroys a motor and the robot enters *Broken*. When *broken* the robot is beyond repair and is sad.

(a) Verify that the transition probabilities form a valid distribution for each state-action pair. Which state is *absorbing* (terminal)?

Row sums: Patrol: $0.7 + 0.3 + 0 + 0 = 1$, Charge: $1 + 0 + 0 + 0 = 1$, Vacuum: $0.6 + 0.35 + 0.05 + 0 = 1$, Explore: $0.4 + 0.3 + 0.3 + 0 = 1$, Reset: $0 + 1 + 0 + 0 = 1$, Force: $0.5 + 0 + 0 + 0.5 = 1$, Wait: $0 + 0 + 0 + 1 = 1$.

Broken is the absorbing (terminal) state, it transitions to itself with probability 1 and offers no escape. The robot is gone. Forever. Sad.

(b) For each non-absorbing state, identify the "safe" action and the "risky" action. Justify using the transition model. For Jammed specifically, note that Force has a positive immediate reward (+1) but non-zero Broken probability. Reset has a negative reward (−5) but a guaranteed non-catastrophic outcome. Why might Reset still be preferred at high γ?

- Clean: safe = Charge (deterministically stays Clean, $P'(\text{Broken}) = 0$); risky = Patrol (30% chance of landing in Messy).
- Messy: safe = Vacuum (at most 5% chance of Jammed, 0% Broken); risky = Explore (30% chance of Jammed).
- Jammed: safe = Reset (guaranteed non-catastrophic/always lands in Messy); risky = Force (50% chance of Broken, which is a death sentence).

Reset is preferred at high γ because a patient agent (high γ) cares deeply about the future. Broken carries a massive negative value that never stops hurting, $V^*(\text{Broken}) = -200$ at $\gamma = 0.9$. The one-time cost of -5 from Reset is trivial compared to the 50% chance of inheriting -200 from Force. Immediate rewards matter less and less as $\gamma \to 1$.

(c) When $\gamma \to 0$[5] (completely myopic), what action does the agent take from each non-absorbing state? Show the Q-value comparisons that support each choice.

[5] Gen α ipad goblin

When $\gamma \to 0$, all future value terms vanish and $Q(s, a) \approx R(s, a)$:

- Clean: $Q(\text{Clean}, \text{Patrol}) \approx 3$ vs $Q(\text{Clean}, \text{Charge}) \approx 1$ -> Patrol wins.
- Messy: $Q(\text{Messy}, \text{Vacuum}) \approx 5$ vs $Q(\text{Messy}, \text{Explore}) \approx 8$ -> Explore wins.
- Jammed: $Q(\text{Jammed}, \text{Reset}) \approx -5$ vs $Q(\text{Jammed}, \text{Force}) \approx +1$ -> Force wins.

The myopic agent at Jammed picks Force solely for the $+1$ immediate reward, completely ignoring that it has a 50% chance of destroying the robot. Classic.

(d) For Messy, write $Q^*(\text{Messy}, \text{Explore})$ in terms of $V^*(\text{Clean})$, $V^*(\text{Messy})$, and $V^*(\text{Jammed})$. For Jammed, write both $Q^*(\text{Jammed}, \text{Reset})$ and $Q^*(\text{Jammed}, \text{Force})$ in terms of $V^*(\text{Clean})$, $V^*(\text{Messy})$, and $V^*(\text{Broken})$. Argue qualitatively which action dominates at Jammed as $\gamma \to 1$.

$$Q^*(\text{Messy}, \text{Explore}) = 8 + \gamma(0.4 \cdot V^*(\text{Clean}) + 0.3 \cdot V^*(\text{Messy}) + 0.3 \cdot V^*(\text{Jammed})) \quad (3)$$

$$Q^*(\text{Jammed}, \text{Reset}) = -5 + \gamma \cdot V^*(\text{Messy}) \quad (4)$$

$$Q^*(\text{Jammed}, \text{Force}) = 1 + \gamma(0.5 \cdot V^*(\text{Clean}) + 0.5 \cdot V^*(\text{Broken})) \quad (5)$$

As $\gamma \to 1$: $V^*(\text{Broken}) = -\frac{20}{1-\gamma} \to -\infty$. The $0.5 \cdot V^*(\text{Broken})$ term drags $Q^*(\text{Jammed}, \text{Force})$ to $-\infty$. Meanwhile, $Q^*(\text{Jammed}, \text{Reset})$ stays finite (Messy has a real value). Reset dominates completely at high γ.

II Part II: Horizons and Discounting

Q5: A key structural choice in any MDP is whether the agent acts for a fixed number of steps or indefinitely. [6]

[6] 08-MDP-Bellman, Page 5

(a) What is a *finite horizon* MDP, and why may the optimal policy be *non-stationary*? What is an *infinite horizon* MDP, and why is the optimal policy *stationary* in that setting?

A *finite horizon* MDP has the agent act for exactly H steps. The optimal policy can be non-stationary because time to deadline changes the decision calculus, being in state s with 10 steps left warrants different behavior than being in s with 1 step left (near the end you might take risks you'd never take otherwise).

An *infinite horizon* MDP has the agent act indefinitely. The optimal policy is stationary because the situation looks identical at every time step, infinite horizon remains infinite regardless of when you check, so the same state always calls for the same action.

(b) Infinite-horizon MDPs require a discount factor $\gamma \in [0, 1)$. Using the Robot Vacuum MDP from Q4, demonstrate concretely why $\gamma = 1$ causes a problem for two scenarios:

(i) Write the undiscounted total reward for an agent stuck in Broken forever, and for an agent cycling optimally between Clean and Messy forever. Why can these not be compared?

- Stuck in Broken: $-20 + (-20) + (-20) + \ldots = -\infty$.
- Cycling optimally (alternating Patrol and Explore): $+3 + 8 + 3 + 8 + \ldots = +\infty$.

Both sums diverge, one to $-\infty$, one to $+\infty$. The problem is that *any* policy with a repeating positive reward diverges to $+\infty$ and any with a repeating negative reward diverges to $-\infty$. You cannot rank or compare policies since the return of almost every policy is just $\pm\infty$. The comparison has no meaning.

(ii) Now consider an agent that repeatedly uses Force from Jammed. 50% of the time it escapes to Clean (positive total reward), and 50% of the time it reaches Broken (negative total reward). Why does this make the expected undiscounted return of Force undefined when $\gamma = 1$?

The 50% branch leading to Clean eventually produces $+\infty$ total reward (infinite positive cycles). The 50% branch leading to Broken produces $-\infty$. The expected undiscounted return is:

$$0.5 \cdot (+\infty) + 0.5 \cdot (-\infty) = \infty - \infty \tag{6}$$

This is an indeterminate form, mathematically undefined. The agent literally cannot decide whether Force is a good idea.

(c) Discounting is justified two ways. First, as a *preference for sooner rewards*. Second, as a model of *uncertain termination*. Describe both interpretations in your own words.

- *Preference for sooner rewards*: A reward received now is worth more than the same reward received later (consider interest rates or opportunity costs). Discounting by γ^t bakes this in, each time step into the future, rewards are worth γ times less. This matches how real agents (and companies, and people) actually behave.
- *Uncertain termination*: Suppose at each time step there's a fixed probability $(1-\gamma)$ that the interaction ends permanently (the robot gets turned off, the game ends, the episode terminates). Then γ is the survival probability per step. The expected reward of a future payoff is naturally weighted by the probability that you're still around to receive it, which is exactly γ^t after t steps.

(d) Write a closed-form expression for $V^*(\text{Broken})$ in terms of γ. Show the derivation using the Bellman equation for the absorbing state. What happens to this value as $\gamma \to 1$, and why does this matter for Jammed's Force action?

Broken has only Wait available and self-loops with reward -20:

$$V^*(\text{Broken}) = -20 + \gamma \cdot V^*(\text{Broken}) \tag{7}$$

$$V^*(\text{Broken})(1-\gamma) = -20 \tag{8}$$

$$V^*(\text{Broken}) = \frac{-20}{1-\gamma} \tag{9}$$

At $\gamma = 0.9$: $V^*(\text{Broken}) = -\frac{20}{0.1} = -200$.

As $\gamma \to 1$: $V^*(\text{Broken}) \to -\infty$. This matters enormously for Force at Jammed: $Q^*(\text{Jammed}, \text{Force})$ contains a $0.5\gamma \cdot V^*(\text{Broken})$ term, which diverges to $-\infty$. The more the agent cares about the future, the more catastrophic landing in Broken becomes.

(e) The discount parameter γ directly controls the agent's patience. For the Jammed state specifically:

- When $\gamma \to 0$: Write the approximate values of $Q(\text{Jammed}, \text{Reset})$ and $Q(\text{Jammed}, \text{Force})$ (the future terms vanish). Which action does the myopic agent select and why?
- When $\gamma \to 1$: Use your closed-form $V^*(\text{Broken})$ from (d) to write $Q^*(\text{Jammed}, \text{Force})$ explicitly. What happens to this Q-value as $\gamma \to 1$? Compare to $Q^*(\text{Jammed}, \text{Reset})$ and state which action now dominates.

$\gamma \to 0$:

$$Q(\text{Jammed}, \text{Reset}) \approx -5, \quad Q(\text{Jammed}, \text{Force}) \approx +1 \tag{10}$$

Myopic agent picks Force, it only sees the immediate $+1$ vs -5 and has no idea it's playing Russian roulette with the motor.

$\gamma \to 1$:

$$Q^*(\text{Jammed}, \text{Force}) = 1 + \gamma\left(0.5 \cdot V^*(\text{Clean}) + 0.5 \cdot \frac{-20}{1-\gamma}\right) \tag{11}$$

$$= 1 + 0.5\gamma V^*(\text{Clean}) - \frac{10\gamma}{1-\gamma} \tag{12}$$

As $\gamma \to 1$, the $-10\frac{\gamma}{1-\gamma}$ term $\to -\infty$, so $Q^*(\text{Jammed}, \text{Force}) \to -\infty$.

Meanwhile $Q^*(\text{Jammed}, \text{Reset}) = -5 + \gamma V^*(\text{Messy})$ stays finite. Reset completely dominates as $\gamma \to 1$.

(f) Now suppose every non-Broken state receives a *living reward* of $r_L = -1$ on every step, added on top of the action rewards already in the table. How does this change the agents urgency to escape Jammed compared to its urgency to avoid reaching Broken in the first place?

The living reward of -1 applies to every step spent in Clean, Messy, or Jammed, but *not* Broken, which is terminal and earns no living reward. This creates two distinct effects:

Urgency to escape Jammed increases. Each step in Jammed now costs an extra -1 on top of the action reward. The effective reward of Reset becomes $-5 + (-1) = -6$ and Force becomes $+1 + (-1) = 0$. More importantly, every step of delay compounds: the agent bleeds -1 per step regardless of which recovery action it eventually takes, adding time pressure that did not exist before.

Urgency to avoid Broken is unchanged. $V^*(\text{Broken})$ is unaffected because Broken is excluded from the living reward, the per-step penalty never applies there. The catastrophic sink $V^*(\text{Broken}) = -\frac{20}{1-\gamma}$ remains exactly the same.

Net effect is the living reward makes lingering in Jammed more costly, so the agent is pushed to escape sooner. The absolute incentive to avoid Broken does not change, but the relative cost of staying Jammed has risen, making prompt escape via Reset even more attractive relative to gambling on Force.

Q6: Consider the Roomba MDP from Q4 with no discounting ($\gamma = 1$) and a finite horizon of $h = 3$ steps. At each step the agent picks an action and receives the listed reward; after 3 steps the episode ends and no further reward is collected. [7]

[7] Course notes: 08-MDP-Bellman, Page 5

(a) At $h = 0$ steps remaining, $V_0(s) = 0$ for all states. Using the backward induction formula $V_{k(s)} = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + V_{k-1}(s')]$, compute $V_1(s)$ for all four states. Show the Q-values for each available action.

With $V_0 = 0$ everywhere and $\gamma = 1$, each Q-value reduces to the expected immediate reward (all future value terms are zero):

Clean: $Q_1(\text{Patrol}) = 3, \quad Q_1(\text{Charge}) = 1 \rightarrow V_1(\text{Clean}) = 3$ (Patrol)

Messy: $Q_1(\text{Vacuum}) = 5, \quad Q_1(\text{Explore}) = 8 \rightarrow V_1(\text{Messy}) = 8$ (Explore)

Jammed: $Q_1(\text{Reset}) = -5, \quad Q_1(\text{Force}) = 0.5(1) + 0.5(1) = 1 \quad \rightarrow V_1(\text{Jammed}) = 1$ (Force)

Broken: $Q_1(\text{Wait}) = -20 \rightarrow V_1(\text{Broken}) = -20$

(b) Compute $V_2(s)$ using your V_1 values. At Jammed, does the same action win as in step (a)?

Clean:

$$Q_2(\text{Patrol}) = 0.7(3 + V_1(\text{Clean})) + 0.3(3 + V_1(\text{Messy})) = 0.7(6) + 0.3(11) = 4.2 + 3.3 = 7.5 \tag{13}$$

$$Q_2(\text{Charge}) = 1.0(1 + V_1(\text{Clean})) = 1 + 3 = 4 \tag{14}$$

$V_2(\text{Clean}) = 7.5$ (Patrol)

Messy:

$$Q_2(\text{Vacuum}) = 0.6(5 + 3) + 0.35(5 + 8) + 0.05(5 + 1) = 4.8 + 4.55 + 0.3 = 9.65 \tag{15}$$

$$Q_2(\text{Explore}) = 0.4(8 + 3) + 0.3(8 + 8) + 0.3(8 + 1) = 4.4 + 4.8 + 2.7 = 11.9 \tag{16}$$

$V_2(\text{Messy}) = 11.9$ (Explore)

Jammed:

$$Q_2(\text{Reset}) = 1.0(-5 + V_1(\text{Messy})) = -5 + 8 = 3 \tag{17}$$

$$Q_2(\text{Force}) = 0.5(1 + V_1(\text{Clean})) + 0.5(1 + V_1(\text{Broken})) = 0.5(4) + 0.5(-19) = 2 - 9.5 = -7.5 \tag{18}$$

$V_2(\text{Jammed}) = 3$ (Reset), *action changed* from Force to Reset.

Broken: $V_2(\text{Broken}) = -20 + V_1(\text{Broken}) = -20 + (-20) = -40$

No, the winning action at Jammed flipped, with 1 step left Force wins (immediate $+1 > -5$), but with 2 steps left Reset wins because Force's 50% chance of landing in Broken (worth -20 with one step left) drags its Q-value to -7.5.

(c) Compute $V_3(s)$. Does the optimal action at Jammed depend on how many steps remain? What does this concretely illustrate about why finite-horizon optimal policies are *non-stationary*?

Clean:

$$Q_3(\text{Patrol}) = 0.7(3 + 7.5) + 0.3(3 + 11.9) = 0.7(10.5) + 0.3(14.9) = 7.35 + 4.47 = 11.82 \tag{19}$$

$$Q_3(\text{Charge}) = 1 + 7.5 = 8.5 \tag{20}$$

$V_3(\text{Clean}) = 11.82$ (Patrol)

Messy:

$$Q_3(\text{Vacuum}) = 0.6(5 + 7.5) + 0.35(5 + 11.9) + 0.05(5 + 3) = 7.5 + 5.915 + 0.4 = 13.815 \tag{21}$$

$$Q_3(\text{Explore}) = 0.4(8 + 7.5) + 0.3(8 + 11.9) + 0.3(8 + 3) = 6.2 + 5.97 + 3.3 = 15.47 \tag{22}$$

$V_3(\text{Messy}) = 15.47$ (Explore)

Jammed:

$$Q_3(\text{Reset}) = -5 + V_2(\text{Messy}) = -5 + 11.9 = 6.9 \tag{23}$$

$$Q_3(\text{Force}) = 0.5(1 + V_2(\text{Clean})) + 0.5(1 + V_2(\text{Broken})) = 0.5(8.5) + 0.5(-39) = 4.25 - 19.5 = -15.25 \tag{24}$$

$V_3(\text{Jammed}) = 6.9$ (Reset)

Broken: $V_3(\text{Broken}) = -20 + (-40) = -60$

Yes, the optimal action at Jammed depends on how many steps remain: Force is optimal at $k = 1$ but Reset is optimal at $k = 2$ and $k = 3$. This concretely illustrates non-stationarity, the same state (Jammed) calls for a different action depending on *when* in the episode the agent finds itself. In an infinite-horizon MDP, the same state always calls for the same action; here the time-to-end changes what is rational.

III Part III: The Bellman Equation

Q7: Define the *optimal value function* $V^*(s)$ and the *Q-value* (action-value) $Q^*(s,a)$. [8]

[8] Course notes: 08-MDP-Bellman, Page 8

(a) Write the formula for $Q^*(s,a)$ in full, identifying what each term represents.

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')] \tag{25}$$

- $T(s,a,s')$: probability the environment transitions to s' (not the agent's choice)
- $R(s,a,s')$: immediate reward for that transition
- $\gamma V^*(s')$: discounted value of the optimal future starting from s'

(b) Express $V^*(s)$ in terms of $Q^*(s,a)$. Then write the optimal policy $\pi^*(s)$ in terms of $Q^*(s,a)$.

$$V^*(s) = \max_a Q^*(s,a) \tag{26}$$

$$\pi^*(s) = \text{argmax}_a \, Q^*(s,a) \tag{27}$$

(c) Combine your answers to derive the *Bellman equation* for $V^*(s)$. Label which part represents the agent's choice, which part represents the environment's randomness, and which part captures the recursive structure.

$$V^*(s) = \underbrace{\max_a}_{\text{agent's choice}} \underbrace{\sum_{s'} T(s,a,s')}_{\text{environment's randomness}} \Big[R(s,a,s') + \underbrace{\gamma V^*(s')}_{\text{recursive structure}} \Big] \tag{28}$$

The agent picks the action ($\max_a$), the environment rolls the dice ($\sum T$), and the future value of the next state feeds back into the current value (γV^*).

Q8: Using the Robot Vacuum MDP from Q4 with $\gamma = 0.9$, perform two rounds of value iteration starting from $V^0(s) = 0$ for all states. The update rule is $V^{k+1}(s) = \max_a \sum_{s'} T(s,a,s') \big[R(s,a,s') + \gamma V^{k(s')} \big]$. [9]

[9] Course notes: 08-MDP-Bellman, Pages 9–10

(a) Compute $V^1(s)$ for all four states. Note that Broken has only one action (Wait), so no max is needed there. For Clean and Messy, show both Q-values. For Jammed, compare Reset and Force explicitly.

All future values are 0 so every Q^1 is just the immediate reward:

$V^1(\text{Broken}) = -20 + 0.9 \cdot 0 = -20$

Clean: $Q^1(\text{Patrol}) = 3 + 0.9(0) = 3$, $Q^1(\text{Charge}) = 1 + 0.9(0) = 1$ $\rightarrow$ $V^1(\text{Clean}) = 3$ (Patrol)

Messy: $Q^1(\text{Vacuum}) = 5, \quad Q^1(\text{Explore}) = 8 \rightarrow V^1(\text{Messy}) = 8$ (Explore)

Jammed: $Q^1(\text{Reset}) = -5, \quad Q^1(\text{Force}) = 1 \rightarrow V^1(\text{Jammed}) = 1$ (Force)

(b) Compute $V^2(s)$ for all four states. Again show all Q-values at each non-absorbing state. *Hint: notice how* $V^1(\text{Broken}) = -20$ *now feeds into* $Q^2(\text{Jammed}, \text{Force})$ *via the* $0.5 \cdot V^1(\text{Broken})$ *term.*

$V^2(\text{Broken}) = -20 + 0.9(-20) = -38$

Clean:

$$Q^2(\text{Patrol}) = 3 + 0.9(0.7 \cdot 3 + 0.3 \cdot 8) = 3 + 0.9(2.1 + 2.4) = 3 + 4.05 = 7.05 \tag{29}$$

$$Q^2(\text{Charge}) = 1 + 0.9(1.0 \cdot 3) = 1 + 2.7 = 3.7 \tag{30}$$

$V^2(\text{Clean}) = 7.05$ (Patrol)

Messy:

$$Q^2(\text{Vacuum}) = 5 + 0.9(0.6 \cdot 3 + 0.35 \cdot 8 + 0.05 \cdot 1) = 5 + 0.9(1.8 + 2.8 + 0.05) = 5 + 4.185 = 9.185 \tag{31}$$

$$Q^2(\text{Explore}) = 8 + 0.9(0.4 \cdot 3 + 0.3 \cdot 8 + 0.3 \cdot 1) = 8 + 0.9(1.2 + 2.4 + 0.3) = 8 + 3.51 = 11.51 \tag{32}$$

$V^2(\text{Messy}) = 11.51$ (Explore)

Jammed:

$$Q^2(\text{Reset}) = -5 + 0.9(1.0 \cdot 8) = -5 + 7.2 = 2.2 \tag{33}$$

$$Q^2(\text{Force}) = 1 + 0.9(0.5 \cdot 3 + 0.5 \cdot (-20)) = 1 + 0.9(1.5 - 10) = 1 - 7.65 = -6.65 \tag{34}$$

$V^2(\text{Jammed}) = 2.2$ (Reset)

(c) At Jammed, which action did the max select in round 1? Which does it select in round 2? The answer changes, explain intuitively why in terms of what $V^1(\text{Broken})$ contributes to Force's Q-value once it is no longer zero.

Round 1: Force ($1 > -5$). Round 2: Reset ($2.2 > -6.65$).

In round 1, all future estimates were 0, so $Q^1(\text{Force}) = +1$ looked great (only the immediate reward matters). In round 2, $V^1(\text{Broken}) = -20$ is plugged in: the $0.5 \cdot 0.9 \cdot (-20) = -9$ term drags Force down to -6.65. The algorithm has "learned" that half the time Force leads somewhere terrible, and that information propagates backward.

Q9: Now solve for the *true* $V^*(s)$ directly as a linear system. The optimal policy from Q8 (once estimates settled) is $\pi^* = \{\text{Clean} : \text{Patrol}, \text{Messy} : \text{Explore}, \text{Jammed} : \text{Reset}\}$. Because the policy is known, substitute the chosen action at each state and drop the max(this turns the recursive Bellman equation into a solvable linear system). [10]

[10] Course notes: 08-MDP-Bellman, Page 10

(a) Solve for $V^*(\text{Broken})$. Broken is termial with only Wait available, write $V^*(\text{Broken}) = R + \gamma \cdot V^*(\text{Broken})$, then solve for $V^*(\text{Broken})$ in terms of γ. Plug in $\gamma = 0.9$.

$$V^*(\text{Broken}) = -20 + \gamma V^*(\text{Broken}) \tag{35}$$

$$V^*(\text{Broken})(1-\gamma) = -20 \tag{36}$$

$$V^*(\text{Broken}) = \frac{-20}{1-\gamma} \tag{37}$$

At $\gamma = 0.9$: $V^*(\text{Broken}) = -\frac{20}{0.1} = -200$.

(b) Write the Bellman equation for $V^*(\text{Jammed})$ under Reset. Reset transitions deterministically to Messy, so Broken does *not* appear here. You will get: $V^*(\text{Jammed}) = -5 + 0.9 \cdot V^*(\text{Messy})$. [11]

[11] *Don't this solve yet*, keep this as an expression; you'll substitute it in part (c)

Under Reset, $T(\text{Jammed}, \text{Reset}, \text{Messy}) = 1$:

$$V^*(\text{Jammed}) = -5 + 0.9 \cdot V^*(\text{Messy}) \tag{38}$$

(remember this, it's needed for the next parts.)

(c) Write the Bellman equation for $V^*(\text{Clean})$ under Patrol. Collect $V^*(\text{Clean})$ on the left to get the form: $\alpha \cdot V^*(\text{Clean}) = c_1 + \beta \cdot V^*(\text{Messy})$. (Identify α, c_1, β.)

Under Patrol, $T(\text{Clean}, \text{Patrol}, \text{Clean}) = 0.7$ and $T(\text{Clean}, \text{Patrol}, \text{Messy}) = 0.3$:

$$V^*(\text{Clean}) = 3 + 0.9(0.7 \cdot V^*(\text{Clean}) + 0.3 \cdot V^*(\text{Messy})) \tag{39}$$

$$V^*(\text{Clean}) = 3 + 0.63V^*(\text{Clean}) + 0.27V^*(\text{Messy}) \tag{40}$$

$$0.37V^*(\text{Clean}) = 3 + 0.27V^*(\text{Messy}) \tag{41}$$

So $\alpha = 0.37$, $c_1 = 3$, $\beta = 0.27$.

(d) Write the Bellman equation for $V^*(\text{Messy})$ under Explore. The Explore transitions go to Clean (40%), Messy (30%), and Jammed (30%). Substitute your expression from (b) to eliminate $V^*(\text{Jammed})$, then collect $V^*(\text{Messy})$ on the left. You should arrive at: $\delta \cdot V^*(\text{Messy}) = c_2 + \varepsilon \cdot V^*(\text{Clean})$. (Identify δ, c_2, ε.)

Under Explore:

$$V^*(\text{Messy}) = 8 + 0.9(0.4V^*(\text{Clean}) + 0.3V^*(\text{Messy}) + 0.3V^*(\text{Jammed})) \quad (42)$$

Substitute $V^*(\text{Jammed}) = -5 + 0.9V^*(\text{Messy})$:

$$V^*(\text{Messy}) = 8 + 0.9(0.4V^*(\text{Clean}) + 0.3V^*(\text{Messy}) + 0.3(-5 + 0.9V^*(\text{Messy})) \quad (43)$$

$$= 8 + 0.9(0.4V^*(\text{Clean}) + 0.57V^*(\text{Messy}) - 1.5) \quad (44)$$

$$= 8 + 0.36V^*(\text{Clean}) + 0.513V^*(\text{Messy}) - 1.35 \quad (45)$$

$$0.487V^*(\text{Messy}) = 6.65 + 0.36V^*(\text{Clean}) \quad (46)$$

So $\delta = 0.487$, $c_2 = 6.65$, $\varepsilon = 0.36$.

(e) You now have two equations in $V^*(\text{Clean})$ and $V^*(\text{Messy})$ from (c) and (d). Solve this 2×2 linear system (substitute one into the other). Then use (b) to find $V^*(\text{Jammed})$.

From (c): $V^*(\text{Clean}) = \frac{3+0.27M}{0.37}$ where $M = V^*(\text{Messy})$.

Sub into (d):

$$0.487M = 6.65 + \frac{0.36}{0.37}(3 + 0.27M) \quad (47)$$

$$0.487M = 6.65 + 2.919 + 0.2627M \quad (48)$$

$$0.2243M = 9.569 \quad (49)$$

$$M = V^*(\text{Messy}) \approx 42.66 \quad (50)$$

$$V^*(\text{Clean}) = \frac{3 + 0.27 \times 42.66}{0.37} = \frac{14.52}{0.37} \approx 39.24 \quad (51)$$

$$V^*(\text{Jammed}) = -5 + 0.9 \times 42.66 \approx -5 + 38.39 = 33.39 \quad (52)$$

(f) Verify: compute $Q^*(\text{Jammed}, \text{Force})$ and $Q^*(\text{Messy}, \text{Vacuum})$ using your exact values. Confirm Reset beats Force at Jammed and Explore beats Vacuum at Messy.

$$Q^*(\text{Jammed}, \text{Force}) =$$
$$1 + 0.9(0.5 \times 39.24 + 0.5 \times (-200)) =$$
$$1 + 0.9(19.62 - 100) \approx -71.34 \quad (53)$$

$V^*(\text{Jammed}) \approx 33.39 \gg -71.34$ -> Reset wins

$$Q^*(\text{Messy}, \text{Vacuum}) =$$
$$5 + 0.9(0.6 \times 39.24 + 0.35 \times 42.66 + 0.05 \times 33.39) \quad (54)$$

$$= 5 + 0.9(23.54 + 14.93 + 1.67) = 5 + 0.9 \times 40.14 \approx 41.13 \quad (55)$$

$V^*(\text{Messy}) \approx 42.66 > 41.13$ -> Explore wins

Q10: The structure of the Bellman equation should look familiar from earlier in the course. [12]

(a) Identify the correspondence between the Bellman equation and an expectimax tree. What do the agent (square) nodes correspond to? What do the Q-state (circle) nodes correspond to?

- Agent (square) nodes $\leftrightarrow$ the $\max_a$ in the Bellman equation: the agent chooses which action to take.
- Q-state (circle) nodes $\leftrightarrow$ the $\sum_{\{s'\}} T(s, a, s')[R + \gamma V^*(s')]$ part: the expected value over random outcomes of a committed action.

The Bellman equation is essentially a depth-2 expectimax tree (max over actions, then expectation over transitions), applied at every state simultaneously.

(b) Expectimax expands a tree from a single root, which cannot handle cycles without unrolling into an infinite recursion. How does the Bellman equation handle the same problem? What makes it more efficient?

The Bellman equation treats $V^*(s)$ as a variable, not something you expand into a tree. Cycles in the state graph just become self-referential equations, they're resolved by iterative Bellman updates (value iteration) or by solving the linear system directly (policy iteration). Either way, each state is represented once, not expanded infinitely. Expectimax applied naively to a cyclic graph would recurse forever; the Bellman approach breaks the cycle by treating future values as estimates to be updated, not subtrees to be computed.

[12] Course notes: 08-MDP-Bellman, Page 11

IV Part IV: Value and Policy Iteration

Q11: Value iteration finds the optimal value function by repeatedly applying the Bellman equation as an update rule, without ever fixing a policy in advance. [13]

[13] Course notes: 08-MDP-Bellman, Pages 9–10

(a) Write the value iteration update rule $V^{k+1}(s)$ in full. Why does keeping the max inside the update rule allow the algorithm to work without knowing the optimal policy ahead of time?

$$V^{k+1}(s) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^{k(s')} \right] \tag{56}$$

The max lets the update evaluate all available actions and commit to the best one at each step, using the current value estimates. No policy needs to be fixed externally, the algorithm implicitly "tries" every action every round and automatically identifies which one looks best given the current V^k. As V^k converges to V^*, the argmax at each state converges to π^*.

(b) Value iteration is guaranteed to converge for any finite state space when $\gamma < 1$. Explain the contraction argument: why does each round shrink the error between V^k and V^* by at least a factor of γ? What does this say about convergence speed as $\gamma \to 1$?

The Bellman operator $\mathcal{B}$ is a γ-contraction in the max-norm: for any two value functions U and V,

$$\|\mathcal{B}U - \mathcal{B}V\|_\infty \leq \gamma \, \|U - V\|_\infty \tag{57}$$

This follows because the future value terms are scaled by γ in the update, so any error in V^k can contribute at most γ times that error to $V^{\{k+1\}}$. Applying this repeatedly: $\|V^k - V^*\|_\infty \leq \gamma^k \, \|V^0 - V^*\|_\infty$, which shrinks geometrically.

As $\gamma \to 1$, the contraction factor approaches 1, and convergence slows dramatically, you need many more rounds to cut the error by any fixed amount. Near $\gamma = 1$ you're waiting a very long time for things to settle down.

(c) After convergence, how do you extract the optimal policy π^* from the converged values V^*? Write the formula. Why does this work, even though π^* was never explicitly tracked during the iterations?

$$\pi^*(s) = \text{argmax}_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')] \tag{58}$$

It works because V^* already encodes optimal future behavior everywhere. Looking one step ahead from fully-converged values gives exactly the same result as looking infinitely far ahead, the values "know" the future. The argmax at each state using V^* is guaranteed to be the optimal action.

(d) Consider the relationship between value convergence and policy convergence. In the Roomba MDP from Q4, the action selected at Jammed flipped between rounds 1 and 2 of your value iteration (Q8c). Does the policy need to be fully stable for the values to be correct? Explain what "the relative ordering of actions is often correct early on" means in practice.

No, the values don't need a stable policy to be correct; value convergence and policy convergence are separate things. Values converge asymptotically ($V^k \to V^*$), but the implicit policy (argmax of V^k) often stabilizes much earlier.

"Relative ordering is often correct early on" means that even when V^k still has numerical errors relative to V^*, the best action at each state is already the right one. In the Roomba example, by round 2 the policy at all three non-absorbing states had already settled to its final form ($\pi^* = \{\text{Clean} : \text{Patrol}, \text{Messy} : \text{Explore}, \text{Jammed} : \text{Reset}\}$), even though V^2 values were far from the true V^* values (e.g., $V^2(\text{Messy}) = 11.51$ vs $V^*(\text{Messy}) \approx 42.66$). In practice this means you can often stop value iteration early once the policy stops changing, even if the values haven't fully converged.

Q12: Policy iteration is an alternative algorithm that alternates between two steps: *policy evaluation* and *policy improvement*. [14] Describe both steps precisely:

[14] Course notes: 08-MDP-Bellman, Pages 12–13

- *Policy evaluation*: Given a fixed policy π, what system of equations do you solve? Why is there no max in this system, and what does that allow you to do?

Given fixed π, solve the linear system:

$$V^{\pi(s)} = \sum_{s'} T(s, \pi(s), s') \left[R(s, \pi(s), s') + \gamma V^{\pi(s')} \right] \quad \forall s \qquad (59)$$

There is no max because the policy is fixed, every state has exactly one prescribed action, so the "choice" is already made. This turns the Bellman equations into a system of $|S|$ linear equations in $|S|$ unknowns (no nonlinearities from the max), which can be solved exactly via Gaussian elimination or matrix inversion rather than iteratively.

- *Policy improvement*: Given the values V^π from evaluation, how do you compute a new policy π'? Write the formula. What does it mean if $\pi' = \pi$?

$$\pi'(s) = \text{argmax}_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi(s')} \right] \qquad (60)$$

If $\pi' = \pi$, then no state has a better action under the current value estimates, the policy is optimal (convergence criterion). You're done.

Q13: Using the Robot Vacuum MDP from Q4 with $\gamma = 0.9$, run one full iteration of policy iteration starting from the all-safe policy $\pi^0 = \{\text{Clean} : \text{Charge}, \text{Messy} : \text{Vacuum}, \text{Jammed} : \text{Reset}\}$.

(a) *Policy evaluation*: Broken is absorbing with $V^*(\text{Broken}) = -200$. For the remaining states, write and solve the three linear equations under π^0. Show how you eliminate variables.

Under π^0:

Clean (Charge): $V(\text{Clean}) = 1 + 0.9V(\text{Clean})$ -> $0.1V(\text{Clean}) = 1$ -> $V(\text{Clean}) = 10$

Jammed (Reset): $V(\text{Jammed}) = -5 + 0.9V(\text{Messy})$.

Messy (Vacuum): $V(\text{Messy}) = 5 + 0.9(0.6 \times 10 + 0.35V(\text{Messy}) + 0.05V(\text{Jammed}))$

Substitute $V(\text{Jammed}) = -5 + 0.9V(\text{Messy})$:

$$V(\text{Messy}) = 5 + 0.9(6 + 0.35V(\text{Messy}) + 0.05(-5 + 0.9V(\text{Messy})) \quad (61)$$

$$= 5 + 0.9(5.75 + 0.395V(\text{Messy})) \quad (62)$$

$$= 10.175 + 0.3555V(\text{Messy}) \quad (63)$$

$$0.6445V(\text{Messy}) = 10.175 \quad (64)$$

$$V(\text{Messy}) \approx 15.79 \quad (65)$$

$V(\text{Jammed}) = -5 + 0.9 \times 15.79 \approx 9.21$

Summary: $V^{\pi^0}(\text{Clean}) = 10$, $V^{\pi^0}(\text{Messy}) \approx 15.79$, $V^{\pi^0}(\text{Jammed}) \approx 9.21$, $V^{\pi^0}(\text{Broken}) = -200$.

(b) *Policy improvement*: Using your V^{π^0} values, compute all Q-values for every non-absorbing state. Fill in the table:

State	**Action**	$Q(s, a)$ under V^{π^0}
Clean	Patrol	$3 + 0.9(0.7 \times 10 + 0.3 \times 15.79) \approx 13.56$
Clean	Charge	$1 + 0.9 \times 10 = 10$
Messy	Vacuum	≈ 15.79 (self-consistent with policy eval)
Messy	Explore	$8 + 0.9(0.4 \times 10 + 0.3 \times 15.79 + 0.3 \times 9.21) \approx 18.35$
Jammed	Reset	$-5 + 0.9 \times 15.79 \approx 9.21$
Jammed	Force	$1 + 0.9(0.5 \times 10 + 0.5 \times (-200)) = -84.5$

Improved policy $\pi^1 = \{\text{Clean} : \text{Patrol}, \text{Messy} : \text{Explore}, \text{Jammed} : \text{Reset}\}$.

This differs from π^0 at Clean (Charge -> Patrol) and Messy (Vacuum -> Explore). Jammed stays on Reset, Force is -84.5 vs Reset's 9.21, not even close.

(c) If $\pi^1 \neq \pi^0$, one more round of policy evaluation is required. Assuming the policy has stabilized after this round, how would you verify that π^1 is optimal without running another improvement step?

Compute the exact values V^{π^1} for the new policy (solve the linear system under Patrol/Explore/Reset). Then for every non-absorbing state, verify that the current policy action achieves the maximum Q-value (ie check that no other action would be preferred)

$$\pi^1(s) = \text{argmax}_a \sum_{s'} T(s,a,s') \Big[R(s,a,s') + \gamma V^{\pi^1}(s') \Big] \quad \forall s \quad (66)$$

If the improvement step would reproduce π^1 unchanged, the policy is optimal. In this MDP, π^1 is in fact π^* (it matches the policy we solved for in Q9).

Q14: Compare and contrast value iteration and policy iteration. [15]

(a) Both algorithms are guaranteed to converge to the same V^* and π^*. Describe the key structural difference in *how* each algorithm reaches that result. In one sentence each: what does value iteration commit to at each step, and what does policy iteration commit to?

[15] Course notes: 08-MDP-Bellman, Page 13

Value iteration commits to a new *value function* estimate at each step, it updates every $V(s)$ using a max over all actions, without ever pinning down a specific policy.

Policy iteration commits to a specific *policy* at each step, it fully evaluates that policy (exactly), then improves it, alternating between the two.

(b) Policy iteration is often said to converge in fewer *iterations* than value iteration, even though each iteration of policy iteration is more expensive. Explain why:

- Policy iteration converges in a finite number of iterations (hint: what is the bound, and why must it terminate?).
- Value iteration converges asymptotically but may take many rounds before V^k is close enough to V^* to extract the correct policy.

Policy iteration must terminate in at most $|A|^{|S|}$ iterations (the number of distinct policies). Each improvement step strictly improves the policy (or terminates), so no policy is ever repeated, the algorithm can't cycle. With a finite set of policies, it must halt.

Value iteration never fixes a policy, it updates real-valued estimates that converge asymptotically. Each round shrinks the error by γ, but when γ is close to 1 this is slow, and many rounds may pass before the implicit policy (argmax from V^k) stabilizes. You might need hundreds of rounds to get close enough to V^* that the argmax produces the correct action everywhere.

(c) Value iteration requires knowing T and R upfront. What algorithmic family handles the case where the agent must *learn* these from experience instead of having them given?

Reinforcement Learning (RL): the agent learns T and/or R from experience rather than being handed a model.

V Part V: Reinforcement Learning

Q15: Reinforcement learning addresses the setting where the agent still has an MDP but does not know T or R. [16] List the four components of the MDP that still exist in the RL setting. What is the one critical difference from the planning setting?

[16] Course notes: 09-Reinforcement-Learning, Page 1

The four components that still exist: S (states: the agent can observe its current state), A (actions: the agent knows what it can do), γ (discount factor: a design choice baked in by the engineer), and r (observed rewards: the agent sees the reward signal after each transition).

The critical difference: T and R are unknown. The agent cannot compute $\sum_{\{s'\}} T(s, a, s')[...]$ because it doesn't have access to the transition model or the full reward function. It must learn about the world by actually doing stuff and seeing what happens.

Q16: Distinguish *offline* learning from *online* learning. In your own words illustrate the difference? Which mode do value iteration and policy iteration belong to, and why?

Offline learning separates the "learning" phase from the "acting" phase, you collect data (or are given a model), learn from it, and then deploy. Online learning interleaves learning and acting, every action produces new experience that immediately informs the next decision.

Offline is like studying a textbook before an exam (you acquire knowledge first, then apply it). Online is like learning a new job by actually doing it (you learn and act simultaneously, every day).

Value iteration and policy iteration are offline, they require the complete model T and R to be known in advance. They do all computation before the agent takes a single action in the world.

Q17: What are the three central challenges of online RL. Explain them in your own words.

1. Unknown transitions: The agent doesn't know T, it must figure out what happens when it takes actions through trial and error.
2. Unknown rewards: The agent doesn't have R upfront, it only sees rewards as it goes, and must infer the reward structure from experience.
3. Exploration vs. exploitation: To learn effectively, the agent needs to try different actions (explore), but to earn reward, it should do what it already thinks is best (exploit). Balancing these is the core tension of online RL, exploring too little means missing better strategies, but exploring too much wastes reward on suboptimal actions.

Q18: In model-based RL the agent estimates the MDP from observed experience, then solves it with standard planning. [17]

[17] Course notes: 09-Reinforcement-Learning, Pages 2–3

(a) Define what a *sample* and an *episode* are in this context.

A *sample* is a single observed transition: (s, a, r, s'), the agent was in state s, took action a, received reward r, and ended up in state s'. One step of interaction.

An *episode* is a complete trajectory from the starting state to a terminal state: a sequence of samples $(s_0, a_0, r_0, s_1, a_1, r_1, ..., s_n)$ where s_n is terminal. One run of the game from start to finish.

(b) Describe the three-step model-based RL procedure. Be precise about how the transition counts are converted into a valid distribution and when the discount factor γ enters.

1. Collect samples by acting in the environment (following some policy, possibly random, possibly current best-guess).
2. Estimate the model: compute $\hat{T}(s, a, s') = \text{count}(s, a, s')/\ \text{count}(s, a)$ (normalize transition counts to get a valid probability distribution summing to 1). Similarly estimate $\hat{R}(s, a, s')$ as the average observed reward for each (s, a, s') tuple.
3. Solve the estimated MDP using standard planning (value iteration or policy iteration) with $\hat{T}$ and $\hat{R}$ in place of T and R. γ enters only at this step, it's used by the Bellman equation in the planning algorithm.

The agent then acts under the computed policy, collects more samples, and repeats.

(c) Musty the mustang claims: "When estimating $\hat{T}(s, a, s')$ from experience, you must keep samples grouped by episode, mixing transitions across episodes gives you a corrupted transition function." Is Musty correct? Cite the concept that justifies your answer. Then identify one situation where Musty *would* be right and episode boundaries do matter.

Musty is wrong. Samples can be lumped together freely, there is no need to keep episodes separate. The justification is the Markov property, since $P\left(s_{\{t+1\}} \mid s_t, a_t\right)$ depends only on the current state and action, not on when in the episode the transition occurred or what happened before it, every (s, a, s') observation is an equally valid draw from the same distribution. Pooling all transitions and counting them together gives the correct empirical estimate.

Episode boundaries do matter when computing returns for direct evaluation, since you need to know the full sequence of rewards from each visit to a state through to the end of the episode. They also matter if terminal states need special handling, you should not count the transition from a terminal state back to the start of the next episode as a real transition in the MDP.

Q19: Consider the following four episodes in an MDP with states A, B, C, D, E and terminal T:

Episode	Trace
1	$B \to C, -1 \quad C \to D, -1 \quad D \to T, +10$
2	$B \to C, -1 \quad C \to D, -1 \quad D \to T, +10$
3	$E \to C, -1 \quad C \to D, -1 \quad D \to T, +10$
4	$E \to C, -1 \quad C \to A, -1 \quad A \to T, -10$

(a) From these samples, estimate $\hat{T}(C, \text{down}, D)$ and $\hat{T}(C, \text{down}, A)$. Show your count-based calculation.

From state C with action "down", transitions occur in all four episodes (C is visited in each).

- $C \to D$: episodes 1, 2, 3 -> 3 times.
- $C \to A$: episode 4 -> 1 time.
- Total from (C, down): 4 transitions.

$$\hat{T}(C, \text{down}, D) = \frac{3}{4} = 0.75 \tag{67}$$

$$\hat{T}(C, \text{down}, A) = \frac{1}{4} = 0.25 \tag{68}$$

(b) Estimate $\hat{R}(D, \text{exit}, T)$ and $\hat{R}(A, \text{exit}, T)$.

$D \to T$: observed in episodes 1, 2, 3. Reward each time: $+10$. Average: $\hat{R}(D, \text{exit}, T) = +10$.

$A \to T$: observed in episode 4. Reward: -10. Average: $\hat{R}(A, \text{exit}, T) = -10$.

(c) Given these estimates and assuming $\gamma = 0.9$, what is the model-based agent's optimal policy? (You do not need to solve the full linear system, reason qualitatively from the reward structure.)

Going through D pays $+10$ at terminal; going through A pays -10. With $\gamma = 0.9$ the agent heavily weighs these future rewards. From C, action "down" leads to D (75% chance of $+10$ at terminal) vs A (25% chance of -10), still clearly good in expectation.

Optimal policy: from B or E, go to C (the only option shown). From C, take "down" (expected positive terminal reward dominates). The agent should try to route through D, not A.

Q20: Model-free RL avoids storing a full transition model by learning values directly from samples. [18] With that in mind, what are the three ways to compute an expected value $E[X]$. Write all three formulas and identify which corresponds to the model-based, model-free, and exact approaches.

[18] Course notes: 09-Reinforcement-Learning, Pages 4–5

1. Exact (requires knowing P): $E[X] = \sum_x x \cdot P(X = x)$. Used in planning when the full distribution is known.
2. Model-based estimate (requires estimated $\hat{P}$): $E[X] \approx \sum_x x \cdot \hat{P}(X = x)$. Build $\hat{P}$ from counts, then compute the expectation analytically using the model.
3. Sample average / model-free: $E[X] \approx \frac{1}{N} \sum_{i=1}^{N} x_i$ where x_i are observed samples. No model needed, just average what you see. This is the model-free approach, and it works by the law of large numbers as $N \to \infty$.

Q21: Describe *Direct Evaluation*. What policy does the agent follow, what does it record, and how does it produce $\hat{V}^\pi$? State the key weakness of direct evaluation that motivates TD learning.

In direct evaluation the agent follows a fixed policy π and runs complete episodes to completion. For each episode, it records the actual total discounted return from every state visited. After many episodes, $\hat{V}^{\pi(s)}$ is estimated as the average total return across all episodes that passed through state s.

its weakness is direct evaluation ignores the Markov structure of the MDP. It treats each state independently, the value of s is estimated only from episodes that actually visit s, with no information shared between states. This is very sample-inefficient, especially for states visited rarely. TD learning fixes this by bootstrapping, updating $\hat{V}(s)$ immediately using the observed next state's current estimate, propagating information backward through the MDP without waiting for complete episodes.

Q22: *Temporal-Difference Learning* (TD learning) fixes the weakness you identified in (Q21). Write:

(a) The formula for the *sample value* $V^\pi_{\text{sample}(s)}$ implied by a single transition (s, a, s', r).

$$V^\pi_{\text{sample}(s)} = r + \gamma \hat{V}^{\pi(s')} \tag{69}$$

This is the immediate reward plus the discounted current estimate of the next state, a one-step lookahead bootstrapped from the existing value function.

(b) The exponential moving-average update rule for $\hat{V}^{\pi(s)}$. Identify the *learning rate* α and explain what it controls. What does the update reduce to when $\alpha = 0$? When $\alpha = 1$?

$$\hat{V}^{\pi(s)} \leftarrow (1-\alpha)\hat{V}^{\pi(s)} + \alpha \cdot V^{\pi}_{\text{sample}(s)} \tag{70}$$

$\alpha \in (0, 1]$ is the learning rat, it controls the tradeoff between retaining prior estimates and incorporating new samples. High α trusts recent experience; low α is more conservative and averages over a longer history.

- $\alpha = 0$: no learning at all, the estimate never changes regardless of observed samples.
- $\alpha = 1$: complete overwrite, the estimate is replaced entirely by the most recent sample, discarding all prior experience.

(c) Why does TD learning converge faster than direct evaluation? What structural fact about MDPs does it exploit that direct evaluation ignores?

TD learning exploits the Markov property. Because the next state s' fully summarizes the future, the estimate $\hat{V}(s')$ is a useful proxy for future returns even without completing the episode. Every single transition immediately updates $\hat{V}(s)$, and information propagates backward through the state space step by step, you don't need to wait for the episode to end.

Direct evaluation ignores this structure entirely: it waits for complete episodes and estimates each state's value from scratch, treating different states as unrelated. TD learning shares information across states through the bootstrapped update, converging in far fewer samples.

Q23: Q-learning extends TD learning to learn optimal policies without a fixed policy. [19]

(a) Explain why TD learning cannot be directly used to find an *optimal* policy. What would you have to do each time the policy changes under pure TD learning?

TD learning estimates $\hat{V}^{\pi}$, the value function for the *specific policy* π currently being followed. If you want to improve the policy (like in policy iteration), you'd need to re-run TD learning from scratch every time you change π, because the old $\hat{V}^{\pi}$ estimates are now stale and wrong under the new policy. In an online setting where the policy changes frequently, this is completely impractical.

[19] Course notes: 09-Reinforcement-Learning, Pages 5–6

(b) Write the Q-learning update rule for a transition (s, a, s', r). Label every term.

$$\underbrace{Q(s,a)}_{\text{new estimate}} \leftarrow \underbrace{(1-\alpha)}_{\text{keep old}} \cdot \underbrace{Q(s,a)}_{\text{old estimate}} + \underbrace{\alpha}_{\text{step size}} \cdot \left[\underbrace{r}_{\text{immediate reward}} + \underbrace{\gamma}_{\text{discount}} \underbrace{\max_{a'} Q(s',a')}_{\text{optimal future}} \right]$$

(c) Compare the Q-learning update to the TD learning update. What single structural change makes Q-learning *off-policy* while TD learning is *on-policy*? Where does the max appear, and why does that matter?

TD update: $\hat{V}^{\pi(s)} \leftarrow (1-\alpha)\hat{V}^{\pi(s)} + \alpha\left[r + \gamma \hat{V}^{\pi(s')}\right]$

Q-learning update: $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left[r + \gamma \max_{\{a'\}} Q(s',a')\right]$

The single structural change is the $\max_{\{a'\}}$ in the Q-learning target. TD learning uses $\hat{V}^{\pi(s')}$, the value of the next state under the current policy π, so it only learns about what π would do. Q-learning uses $\max_{\{a'\}} Q(s', a')$, the best possible Q-value at s' regardless of what policy is being followed. This decouples learning from the behavior policy: the agent can wander around doing anything (exploring randomly, following a suboptimal policy) and still accumulate updates that converge to the optimal Q-values.

(d) Once Q-values have converged, how do you extract the policy $\pi(s)$? Write the formula.

$$\pi(s) = \text{argmax}_a \, Q(s,a) \tag{71}$$

Just take the greedy action at each state, whichever action has the highest converged Q-value.

(e) Q-learning is guaranteed to converge to the optimal policy given two conditions. State both conditions. Why does "it does not matter how you select actions" hold in the limit, even if early actions are suboptimal?

1. Every state-action pair (s, a) must be visited infinitely often (sufficient exploration, you can't learn about an action you never try). [20]
2. The learning rate α must satisfy the Robbins-Monro conditions: $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$ (e.g., $\alpha_t = \frac{1}{t}$).

"It does not matter how you select actions" holds because over an infinite horizon, every (s, a) pair accumulates infinitely many updates regardless of the exploration strategy. The finitely many "bad" updates from early suboptimal behavior become negligible against the infinite stream of converging updates. Any exploration strategy that satisfies condition 1 will eventually drive Q-values to Q^*.

[20] Yes, you do have to try the obviously bad action infinitely many times. For science.https://xkcd.com/242/

(f) An agent doing Q-learning in an unknown environment with states A, B, C, terminal T, and actions $\leftarrow$ and $\rightarrow$ observes the following repeating sequence (with $\alpha = 0.5$, $\gamma = 0.9$, all Q-values initialized to 0):

s	**a**	s'	**r**
A	$\rightarrow$	B	0
B	$\rightarrow$	C	0
C	$\leftarrow$	T	1

After which sample does $Q(C, \leftarrow)$ first become nonzero? After which sample does $Q(B, \rightarrow)$ first become nonzero? Trace the update for each.

Number samples sequentially (1 = first $A \rightarrow B$, 2 = first $B \rightarrow C$, 3 = first $C \leftarrow T$, then 4,5,6 on the second pass, etc.).

Sample 1 $(A, \rightarrow, B, 0)$: $Q(A, \rightarrow) \leftarrow 0.5 \cdot 0 + 0.5(0 + 0.9 \cdot 0) = 0$. No change.

Sample 2 $(B, \rightarrow, C, 0)$: $Q(B, \rightarrow) \leftarrow 0.5 \cdot 0 + 0.5(0 + 0.9 \cdot \max_a Q(C, a)) = 0$. No change ($Q(C, \cdot)$ still all zero).

Sample 3 $(C, \leftarrow, T, 1)$: T is terminal so $\max_a Q(T, a) = 0$. $Q(C, \leftarrow) \leftarrow 0.5 \cdot 0 + 0.5(1 + 0.9 \cdot 0) = 0.5$. $Q(C, \leftarrow)$ first becomes nonzero after sample 3.

Sample 4 $(A, \rightarrow, B, 0)$: no change.

Sample 5 $(B, \rightarrow, C, 0)$: $Q(B, \rightarrow) \leftarrow 0.5 \cdot 0 + 0.5(0 + 0.9 \cdot \max_a Q(C, a)) = 0.5(0.9 \cdot 0.5) = 0.225$. $Q(B, \rightarrow)$ first becomes nonzero after sample 5.

The reward signal propagates backward one step per pass through the sequence, it reaches C on pass 1, then reaches B on pass 2.

(g) An agent ran Q-learning for 10,000 steps but Q-values have not converged. Three classmates each suspect a different exploration strategy is the cause. For each diagnosis below, state whether the classmate is correct and explain why in one sentence:

- Musty used a fully *greedy* strategy: always pick $\arg\max_{a'} Q(s, a')$ from the current estimates.
- Jeffery used a *fixed optimal policy* π^*: always take the action known to be best from a separate planner.
- Sam used ε-*greedy* with $\varepsilon = 0.05$: pick randomly 5% of the time, greedily otherwise.

Musty (greedy), Correct diagnosis. A fully greedy strategy never explores: once Q-values tip in any direction, the agent keeps exploiting those estimates and never visits state-action pairs whose current Q-value is not the highest, violating the "every (s, a) visited infinitely often" convergence requirement.

Jeffery (fixed π^*), Correct diagnosis. Following a fixed policy (even an optimal one) covers only the state-action pairs that policy visits; all off-path (s, a) pairs are never sampled, so Q-values for those pairs never update and the algorithm cannot converge globally.

Sam (ε-greedy, $\varepsilon = 0.05$), Incorrect diagnosis. ε-greedy with any fixed $\varepsilon > 0$ guarantees that every action is selected with positive probability at every state on every visit, satisfying the infinite-visitation condition; Q-values are guaranteed to converge to Q^* given appropriate learning rates.

Q24: Your robot has finished collecting experience and the environment is no longer accessible, you have only a fixed dataset of stored episodes. For each algorithm, answer: (1) Can it continue improving using only the stored dataset? (2) Can it produce an optimal policy π^* from that dataset alone?

Algorithm	(1) Improves from fixed dataset?	(2) Produces π^*?
Model-based RL	Yes	Yes
Direct Evaluation	Yes	No
TD Learning	Yes	No
Q-Learning	Yes	Yes

All four algorithms can extract information from a fixed dataset: model-based RL builds $\hat{T}$ and $\hat{R}$ from the stored transitions; direct evaluation and TD learning average returns or bootstrap over stored samples; Q-learning applies its update rule to stored (s, a, r, s') tuples.

Direct evaluation and TD learning converge to V^π, the value function of the *behavioral policy* that generated the data, not the optimal policy, without interacting with the environment they cannot evaluate actions the behavior policy never took, so they cannot identify π^*. Model-based RL builds a full transition model from the data and then runs planning to optimality; Q-learning's $\max_{\{a'\}}$ target bootstraps toward the optimal Q-values regardless of the behavior policy, so both can produce π^* from the same fixed dataset.